Report of design

Problem 1 focused on creating a base class CommissionEmployee and a derived class BasePlusCommissionEmployee. The relationship between them is inheritance, where CommissionEmployee is the base class and BasePlusCommissionEmployee is the derive class.

Both classes include constructors and copy constructors. The copy constructors ensure that object copying works correctly. Setter functions evaluate input. Invalid values (e.g. negative sales or negative commission rate) throw invalid_argument exceptions. The toString() function formats output using ostringstream, making the display of object data clear and readable.

For Task 1.4, the downloaded source code and its output are shown in the figures below.

commission.cpp | testDriver.cpp | Point.h | Square.h

This file does not belong to any project target

```
/*
 testDriver.cpp
 *
 *  Created on: Sep 25, 2025
 *      Author: umroot
 */


#include ...

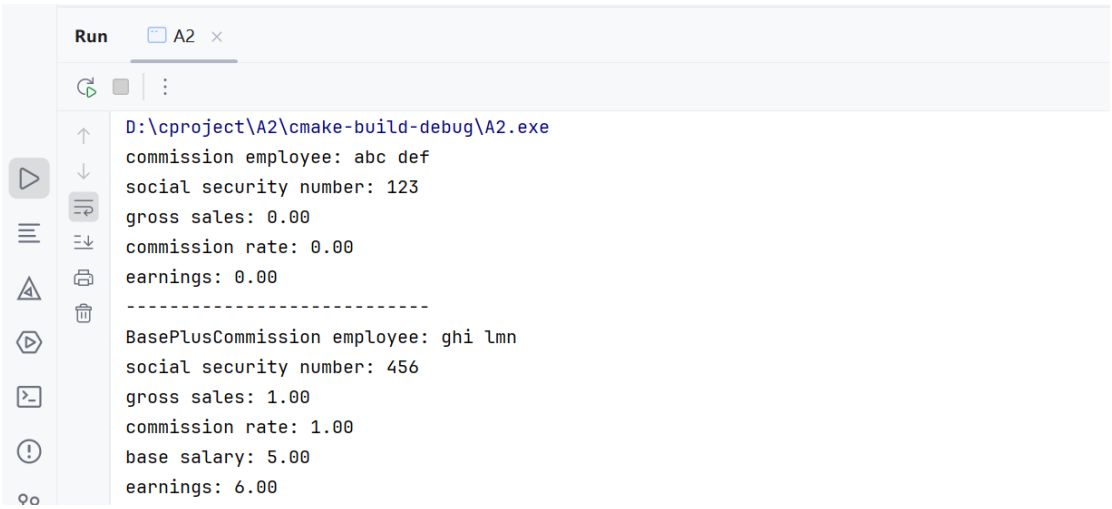
void test(){
    CommissionEmployee cp("abc","def","123",0,0);
    std::cout << cp.toString() << std::endl;

    std::cout << "----------------------------"<<std::endl;

    BasePlusCommissionEmployee bp("ghi","lmn", "456",1,1,5);
    std::cout << bp.toString() << std::endl;

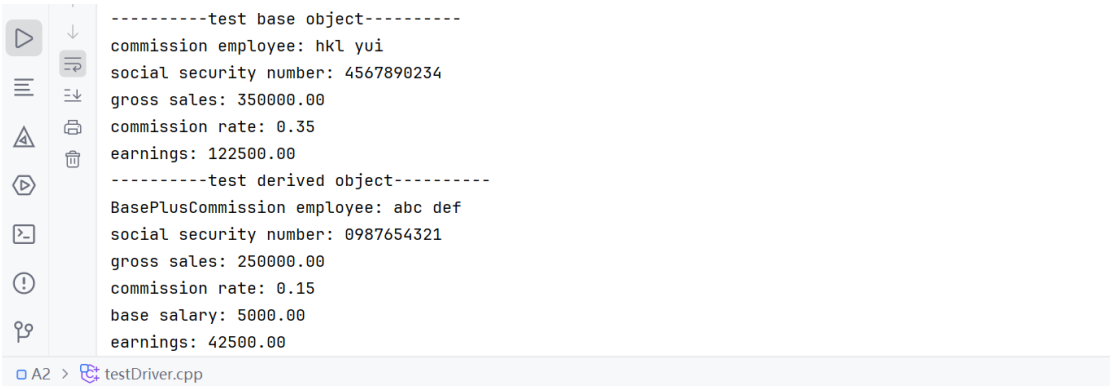

}

int main(){
    test();
}
```



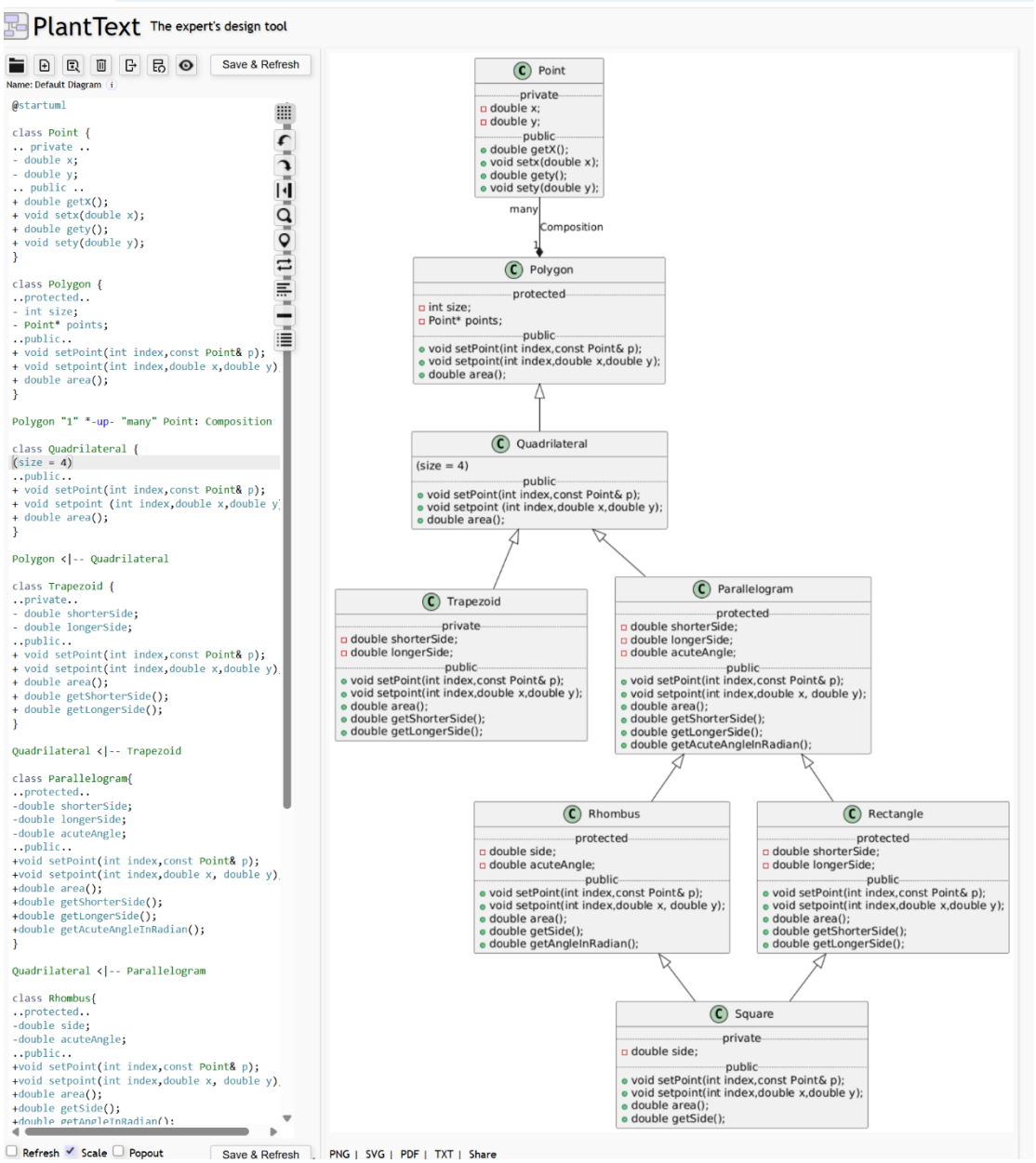
To get output consistent with the assignment description, a new test equation was written, as shown in the figures below.

```
17  void test_4() {
18      try{
19          CommissionEmployee a( "hkl", "yui", "4567890234",350000,0.35);
20          std::cout<<"----------test base object----------"<<std::endl;
21          std::cout << a.toString() << std::endl;
22
23          BasePlusCommissionEmployee b( "abc", "def", "0987654321", 250000, 0.15, 5000);
24          std::cout<<"----------test derived object----------"<<std::endl;
25          std::cout << b.toString() << std::endl;
26      }
27      catch (const invalid_argument& i) {
28          cout<< "Error: " << i.what()<<endl;
29      }
30  }   //Task1.4 Add a new test function to get output consistent with the assignment description.
31
```



A friend function checkGrossSales() is added to access grossSales directly and it can do logical operations outside the class. Header and source files are separated. Each test is in its own function, keeping the code easy to test.

For problem 2, first draw a UML class diagram to represent the classes and relationships, then program four classes including class Point, class Polygon, class Quadrilateral and class Rhombus. The UML diagram is shown below.



The relationship between point and polygon is composition, where multiple points correspond to a single polygon. The relationship between polygon and quadrilateral is inheritance: polygon is the base class, while quadrilateral is the derived class, since quadrilateral is a type of polygon. The relationship between trapezoid and quadrilateral is inheritance, since trapezoid is a type of quadrilateral; the same applies to the relationship between parallelogram and quadrilateral. Rhombus and rectangle are both types of parallelograms, so the relationship remains inheritance. A square is both a rhombus and a rectangle, so it is a derived class of the latter two.

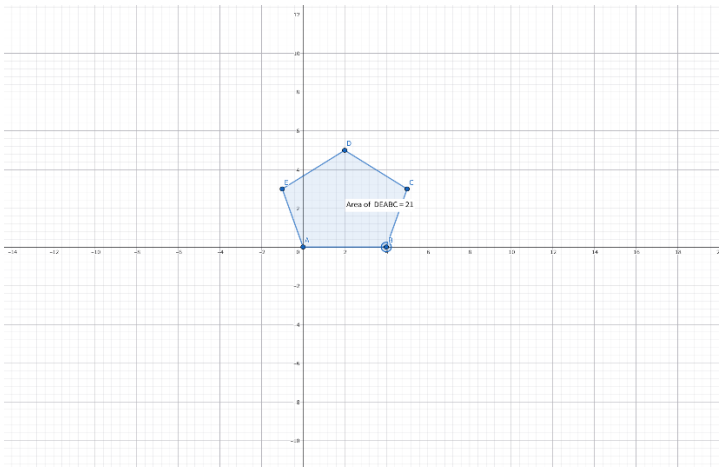
Code reuse is achieved because we use inheritance to create classes. Specifically, Quadrilateral, Rhombus, and Parallelogram all inherit from the Polygon and they share member variables (e.g., points), and common functions from Polygon, such as setPoint, setpoint, and area without rewriting them. This makes the structure cleaner and easier to maintain.

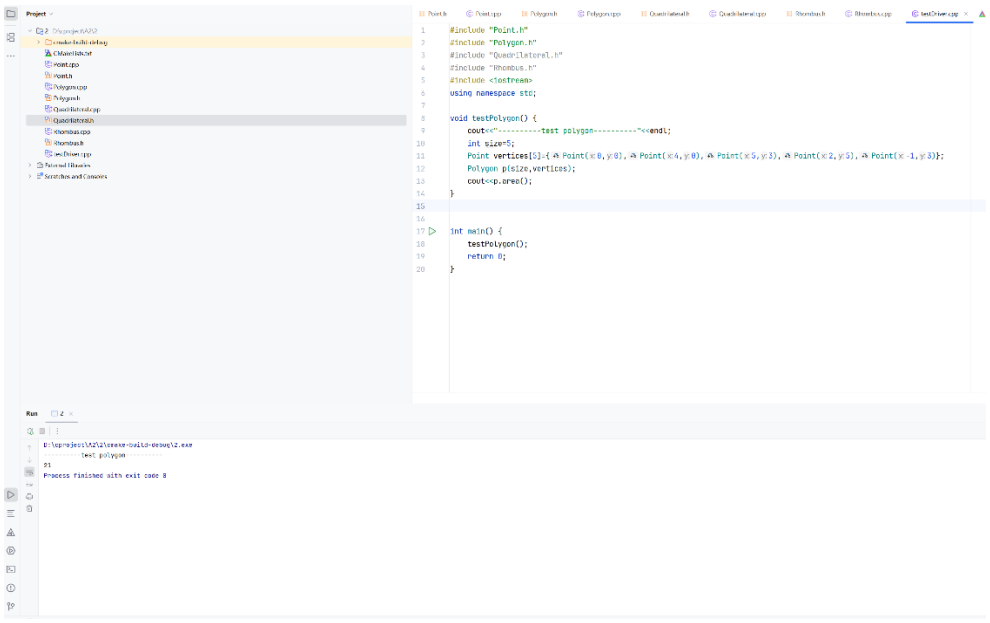
Shared logic is programed in the base class, and in the derived classes, only specific variables like the side length or angle are added. This avoids rewriting the same logic repeatedly.

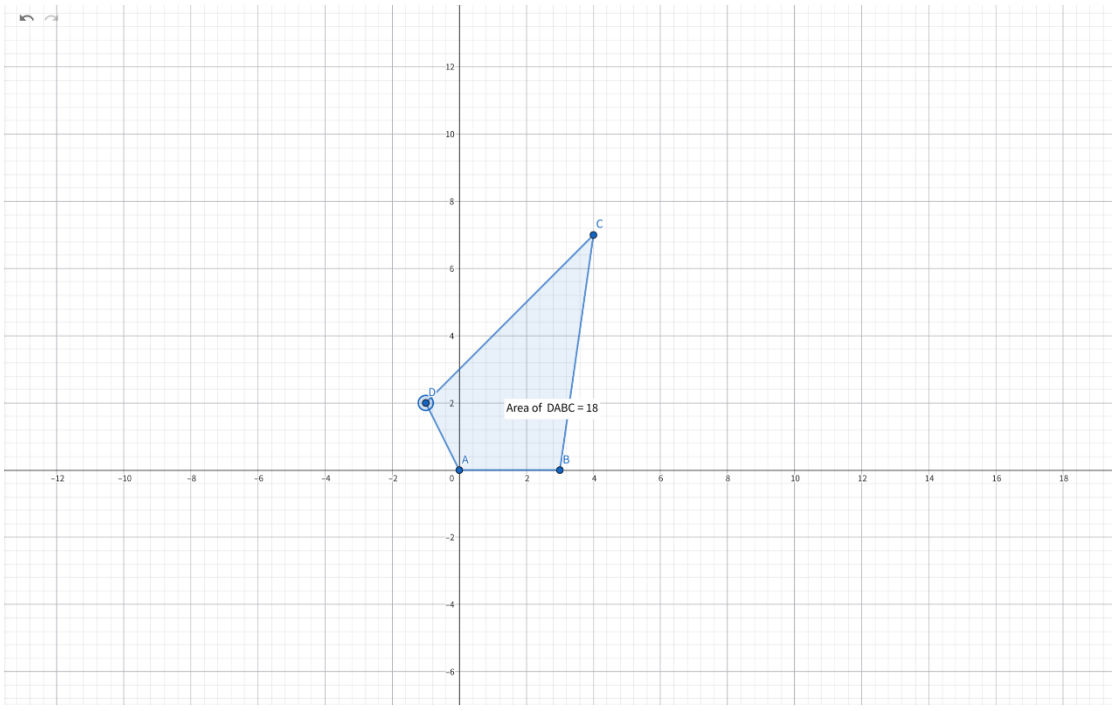
Report of discussion

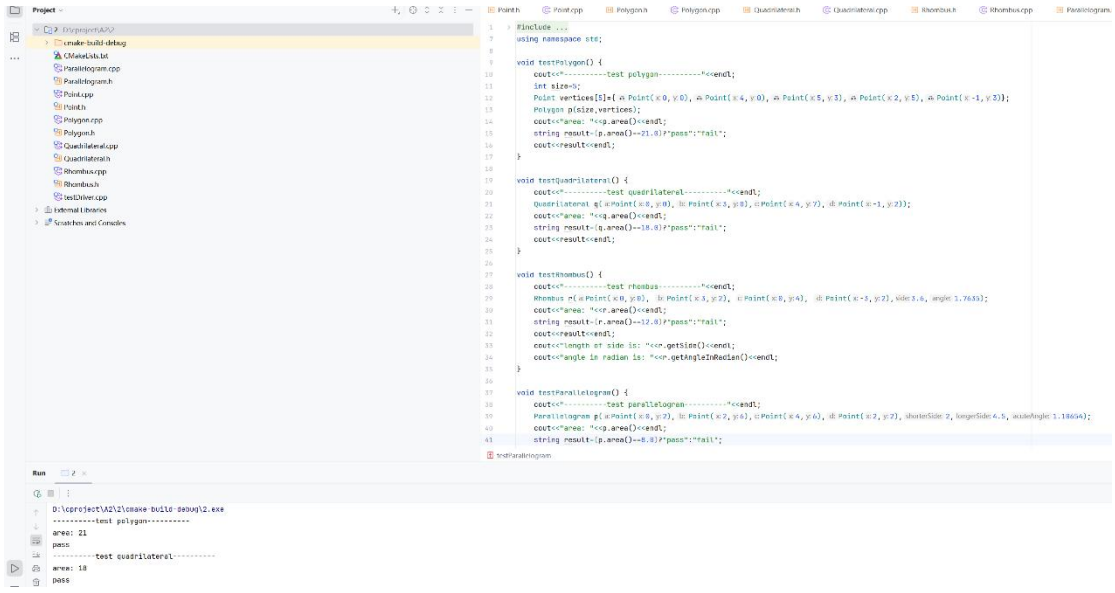
In Task 1.7, the function checkGrossSales was declared as a friend of the CommissionEmployee class. Because BasePlusCommissionEmployee is a subclass of public CommissionEmployee, it inherits all the protected and public members of the base class, including grossSales. Then we created an object of BasePlusCommissionEmployee class, which can be converted into an object of the CommissionEmployee class. Next, since friend functions can access private members of the CommissionEmployee class, this function can run correctly.

For each test function in Problem 2, the geometry shape and the points are created. The shape diagrams and the console screenshots of running a test function are attached below.









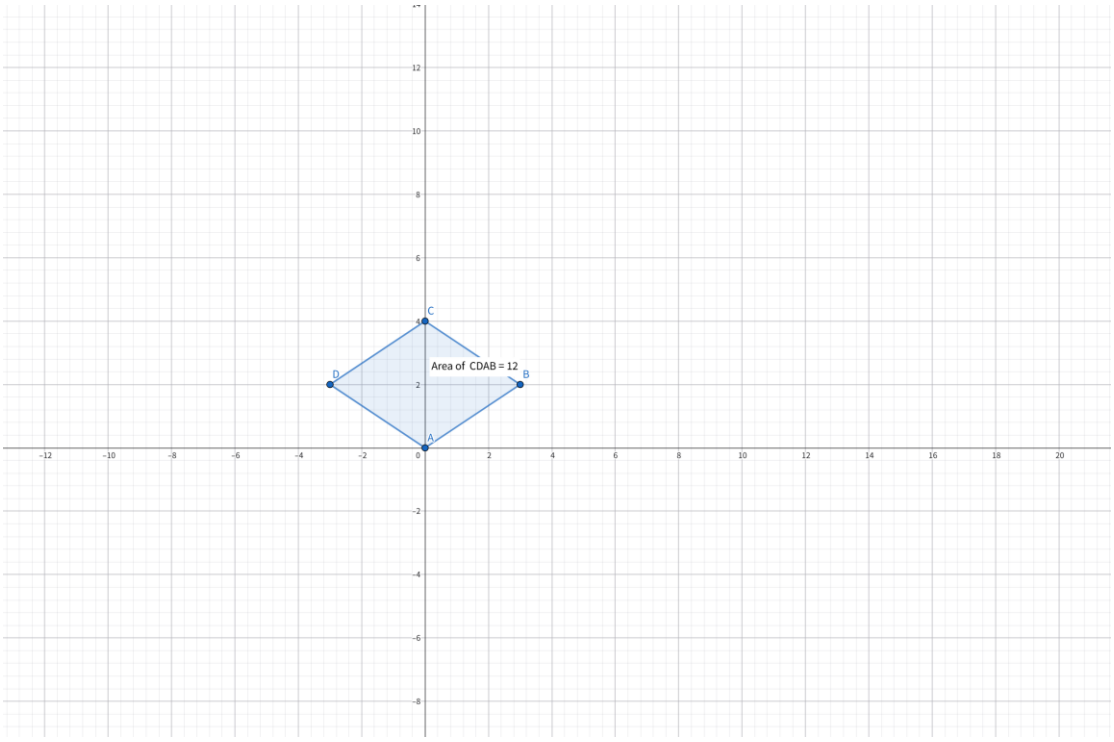
```
Project
Point.h
Point.cpp
Polygon.h
Polygon.cpp
Quadrilateral.h
Quadrilateral.cpp
Rhombus.h
Rhombus.cpp
Parallelogram
#include ...
using namespace std;

void testPolygon() {
    cout<<"----------test polygon----------"<<endl;
    int size=5;
    Point vertices[5]={ Point(0, 0), Point(4, 0), Point(5, 3), Point(2, 5), Point(-1, 3)};
    Polygon p(size,vertices);
    cout<<"area: "<<p.area()<<endl;
    string result=(p.area()==21.0)?"pass":"fail";
    cout<<result<<endl;
}

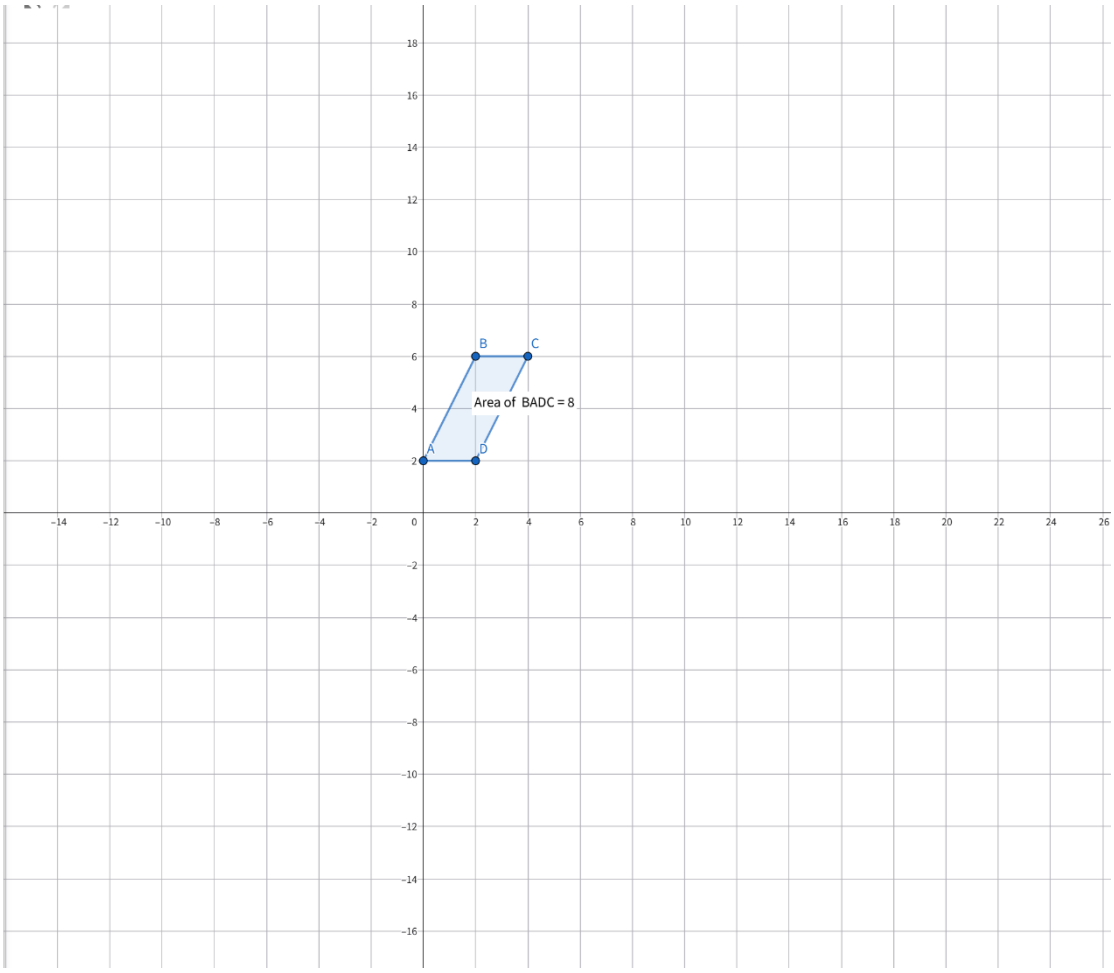
void testQuadrilateral() {
    cout<<"----------test quadrilateral----------"<<endl;
    Quadrilateral q( Point(0, 0), Point(3, 0), Point(4, 7), Point(-1, 2));
    cout<<"area: "<<q.area()<<endl;
    string result=(q.area()==18.0)?"pass":"fail";
    cout<<result<<endl;
}

void testRhombus() {
    cout<<"----------test rhombus----------"<<endl;
    Rhombus r( Point(0, 0),  Point(3, 2),  Point(8, 4),  Point(5, 2), side:3.6, angle:1.76353);
    cout<<"area: "<<r.area()<<endl;
    string result=(r.area()==12.0)?"pass":"fail";
    cout<<result<<endl;
    cout<<"length of side is: "<<r.getSide()<<endl;
    cout<<"angle in radian is: "<<r.getAngleInRadian()<<endl;
}

void testParallelogram() {
    cout<<"----------test parallelogram----------"<<endl;
    Parallelogram p( Point(0, 2), Point(2, 6), Point(4, 6), Point(2, 2), shorterSide: 2, longerSide:4.5, acuteAngle: 1.18694);
    cout<<"area: "<<p.area()<<endl;
    string result=(p.area()==8.0)?"pass":"fail";
testParallelogram
Run
D:\cproject\A2\2\cmake-build-debug\2.exe
----------test polygon----------
area: 21
pass
----------test quadrilateral----------
area: 18
pass
```




```
----------test rhombus----------
area: 12
pass
length of side is: 3.6
angle in radian is: 1.7635
```




```
----------test parallelogram----------
area: 8
pass
length of shorter side is: 2
length of longer side is: 4.5
angle in radian is: 1.10654

Process finished with exit code 0
```

2 > testDriver.cpp